TAREA 4

Implementación física comparativa de la Red Social Pascualina

PostgreSQL · MySQL · SQLite

Juan José Oquendo Sánchez

Institución Universitaria Pascual Bravo

Facultad de Ingeniería

Octubre de 2026

Entrega individual - sin video de sustentación

Resumen ejecutivo

Esta entrega adapta el modelo de la red social estudiantil desarrollado en las tareas previas a tres gestores relacionales. Mantiene las 12 tablas y las 18 claves foráneas del modelo físico de PostgreSQL; extiende la tabla EVENTO con metadatos_contexto, un objeto JSON opcional. El nuevo inventario contiene 58 campos. La modificación incorpora datos semiestructurados sin sustituir las claves ni las relaciones del modelo normalizado.

Se generaron tres scripts DDL equivalentes, un diccionario genérico, tres diccionarios físicos y ejemplos de uso. La estructura SQLite se ejecutó realmente y superó pruebas de integridad; las versiones PostgreSQL y MySQL requieren ejecución y evidencia posterior en sus respectivos motores. Por decisión del autor no se presenta video ni se declara una colaboración grupal inexistente.

1. Antecedentes y alcance

La Tarea 1 definió USUARIO, AREA_ESTUDIO, INTERES_HABILIDAD, PUBLICACION, COMENTARIO, GRUPO y EVENTO; la Tarea 2 transformó relaciones M:N en tablas intermedias (USUARIO_INTERES, CONEXION, MEMBRESIA_GRUPO, REACCION y ASISTENCIA_EVENTO). La Tarea 3 implementó el modelo físico inicial en PostgreSQL. La Tarea 4 conserva esa estructura, adapta sintaxis y controles a tres motores y documenta un atributo semiestructurado añadido a EVENTO.

El caso funcional incluye perfiles, conexiones, publicaciones, grupos, comentarios, reacciones, eventos y confirmación de asistencia. Se descartan nuevas tablas o funciones no solicitadas para mantener la trazabilidad con los entregables previos.

2. Elección de los tres SGBD

Motor	Ventajas relevantes	Limitaciones y riesgos	Motivo de selección
PostgreSQL 13+	JSONB, índices GIN, buen soporte de restricciones y transacciones	Más configuración de servidor y mantenimiento	Continúa la implementación de Tarea 3; motor recomendado
MySQL 8.0.16+ / 8.4	Gran difusión, InnoDB, JSON nativo, herramientas visuales	Incompatibilidades en CHECK con FK; manejo distinto de DATETIME y triggers	Contraste entre dialectos habituales de aplicaciones web
SQLite 3.38+	Archivo portable, instalación simple, SQL y funciones JSON integradas	Sin servidor multiusuario; afinidad de tipos y FK por conexión	Permite comprobar portabilidad y pruebas reproductibles

Las diferencias de JSON y de fecha-hora requieren adaptaciones reales: PostgreSQL utiliza JSONB y TIMESTAMPTZ; MySQL utiliza JSON y DATETIME(6) con convención UTC; SQLite utiliza TEXT con validaciones de forma y funciones JSON. SQLite no incorpora un tipo de almacenamiento JSON independiente.

3. Proceso de implementación paso a paso

1. Revisar el esquema fuente. Se revisaron nombres, claves, dependencias y reglas de la Tarea 3; no se rehizo el modelo desde cero.

2. Extender EVENTO. Se añadió metadatos_contexto, nullable, para una instantánea JSON de contexto.

3. Definir mapeo de tipos. Se tradujeron identificadores, cadenas, fechas y JSON a cada motor.

4. Traducir restricciones. Se mantuvieron PK, UNIQUE, FK y reglas de integridad con particularidades del proveedor.

5. Escribir DDL en orden. Primero catálogos; luego usuario, grupos, publicaciones, comentarios, eventos y tablas puente.

6. Preparar diccionarios. Se registraron los 58 campos con tipo real, límites, nulabilidad, claves, valores y referencias.

7. Probar y documentar. Se ejecutaron scripts y siete rechazos en SQLite; se dejaron instrucciones verificables para los otros motores.

4. Inventario final de las tablas

Tabla	Campos	Clave primaria	Objetivo
AREA_ESTUDIO	3	id_area	Catálogo académico
USUARIO	8	id_usuario	Perfil y área principal
INTERES_HABILIDAD	3	id_interes	Catálogo compartido de intereses
USUARIO_INTERES	3	id_usuario, id_interes	Nivel del usuario por interés
CONEXION	4	id_usuario_origen, id_usuario_destino	Enlace reflexivo y estado
GRUPO	6	id_grupo	Comunidad y creador
MEMBRESIA_GRUPO	4	id_usuario, id_grupo	Rol y fecha en grupo
PUBLICACION	6	id_publicacion	Contenido del muro o grupo
COMENTARIO	5	id_comentario	Respuesta sobre publicación
REACCION	4	id_usuario, id_publicacion	Tipo de reacción por usuario-publicación
EVENTO	9	id_evento	Agenda, organizador y contexto JSON
ASISTENCIA_EVENTO	3	id_usuario, id_evento	Estado de asistencia

La primera columna del diccionario físico indica la tabla y la segunda cada atributo. Las tablas puente usan PK compuestas para impedir duplicación de una asociación. Cada FK representa una relación del modelo conceptual, no una nueva necesidad artificial.

5. Tipos de datos y decisiones físicas

Concepto	PostgreSQL	MySQL	SQLite
Identificador	INTEGER ... IDENTITY	INT AUTO_INCREMENT	INTEGER PRIMARY KEY AUTOINCREMENT
Cadenas cortas	VARCHAR(n)	VARCHAR(n)	TEXT con CHECK(length <= n)
Contenido largo	TEXT	TEXT	TEXT
Fecha de nacimiento	DATE	DATE	TEXT ISO 8601 validado
Instante temporal	TIMESTAMPTZ	DATETIME(6) (UTC convenido)	TEXT UTC con sufijo Z
Metadatos semiestructurados	JSONB	JSON	TEXT validado por JSON1

Para los identificadores se eligió INT/INTEGER (32 bits en PostgreSQL/MySQL) porque el prototipo no prevé superar su capacidad; si el sistema creciera a cientos de millones de usuarios y eventos, correspondería evaluar BIGINT y su efecto en FK e índices. VARCHAR(n) controla límites útiles en nombres y correos, mientras TEXT permite publicaciones y descripciones no acotadas artificialmente. En SQLite el ancho declarado no fuerza un límite, por lo cual se agregan verificaciones de longitud.

El correo institucional tiene UNIQUE en las tres implementaciones. La colación utf8mb4_bin en MySQL mantiene la comparación diferenciando mayúsculas/minúsculas de forma cercana a PostgreSQL y SQLite; se propone normalizar correos a minúsculas en la aplicación. No se supone idéntica semántica lingüística entre motores.

6. Relaciones y acciones referenciales

ON UPDATE CASCADE actualiza las referencias cuando se cambia una clave del padre (situación rara con IDs autogenerados). ON DELETE RESTRICT conserva registros con autores u organizadores responsables; CASCADE elimina dependencias asociativas; SET NULL desasocia contenidos o eventos de un grupo borrado sin eliminarlos.

Tabla hija / campo	Tabla padre	Borrado	Razón
usuario.id_area	area_estudio.id_area	RESTRICT	No perder autor o titular
usuario_interes.id_usuario	usuario.id_usuario	CASCADE	Asociacion dependiente
usuario_interes.id_interes	interes_habilidad.id_interes	RESTRICT	No perder autor o titular
conexion.id_usuario_origen	usuario.id_usuario	CASCADE	Asociacion dependiente
conexion.id_usuario_destino	usuario.id_usuario	CASCADE	Asociacion dependiente
grupo.id_usuario_creador	usuario.id_usuario	RESTRICT	No perder autor o titular
membresia_grupo.id_usuario	usuario.id_usuario	CASCADE	Asociacion dependiente

membresia_grupo.id_grupo	grupo.id_grupo	CASCADE	Asociacion dependiente
publicacion.id_usuario	usuario.id_usuario	RESTRICT	No perder autor o titular
publicacion.id_grupo	grupo.id_grupo	SET NULL	Conservar registro sin grupo
comentario.id_usuario	usuario.id_usuario	RESTRICT	No perder autor o titular
comentario.id_publicacion	publicacion.id_publicacion	CASCADE	Asociacion dependiente
reaccion.id_usuario	usuario.id_usuario	CASCADE	Asociacion dependiente
reaccion.id_publicacion	publicacion.id_publicacion	CASCADE	Asociacion dependiente
evento.id_usuario_organizador	usuario.id_usuario	RESTRICT	No perder autor o titular
evento.id_grupo	grupo.id_grupo	SET NULL	Conservar registro sin grupo
asistencia_evento.id_usuario	usuario.id_usuario	CASCADE	Asociacion dependiente
asistencia_evento.id_evento	evento.id_evento	CASCADE	Asociacion dependiente

Las 18 FK están explicitadas en cada DDL y en las hojas FK_y_acciones de los tres diccionarios. Los valores por defecto para fechas automáticas, los CHECK de dominios (roles, tipos, estados), el orden temporal de EVENTO y la prohibición de conexión reflexiva son controles adicionales.

7. Integración de JSON: escenario IoT

El campo EVENTO.metadatos_contexto almacena una instantánea opcional de datos semiestructurados asociados a un evento universitario: procedencia del dato, geolocalización aproximada, temperatura y humedad reportadas por sensores, e instante de observación. Esto permite enriquecer actividades presenciales sin crear una columna nueva por cada sensor. No implica que existan sensores integrados actualmente.

```
{
 "origen": "sensor_iot",
 "geolocalizacion": {
  "lat": 6.2442,
  "lon": -75.5812
 },
 "ambiente": {
  "temperatura_c": 24.1,
  "humedad_pct": 65
 },
 "instante_medicion": "2026-10-25T18:00:00Z"
}
```

Reglas aplicadas: atributo opcional; si se informa debe ser JSON válido cuya raíz sea un objeto. En PostgreSQL se controla con jsonb_typeof y se declara un índice GIN; en MySQL se utiliza el tipo JSON más JSON_TYPE; en SQLite, TEXT más json_valid y json_type. Los scripts 02 muestran una consulta que recupera la temperatura, adaptada a cada dialecto.

Relación con big data/IoT: un sistema de sensores puede producir señales de alta frecuencia, pero un EVENTO no debe almacenar arreglos sin límite de telemetría. Para cargas masivas convendría una tabla de mediciones con índices y particiones, o un servicio de series de tiempo/almacenamiento NoSQL especializado. El JSON elegido representa solo contexto variable de baja frecuencia.

8. Particularidades y dificultades de cada motor

8.1 PostgreSQL

Se adaptó el DDL de la Tarea 3 y se agregó JSONB opcional, CHECK de raíz objeto e índice GIN para búsquedas por contenido. Se implementan GENERATED ALWAYS AS IDENTITY, TIMESTAMPTZ, PK, UNIQUE y FK con acciones referenciales. Al ejecutar, el usuario debe crear una base vacía y tener permiso para crear el esquema red_social_pascualina. Estas operaciones todavía no fueron probadas contra un servidor PostgreSQL en esta entrega.

8.2 MySQL

Se definió MySQL 8.0.16 o superior (recomendado 8.4), motor InnoDB, utf8mb4 y colación binaria. El CHECK de no autoconexión sobre dos columnas que participan en FK con acciones referenciales no es admisible en MySQL; por ello se sustituyó por dos triggers (INSERT y UPDATE) con SIGNAL SQLSTATE. El script usa DELIMITER, instrucción del cliente MySQL/Workbench. DATETIME(6) conserva el horario literal, por lo que se exige el convenio de aplicación UTC; no equivale a TIMESTAMPTZ. Este DDL aún no se ha probado en un servidor MySQL.

8.3 SQLite

Se implementaron 12 tablas, 18 FK y controles de longitud/fechas/JSON en un archivo local. Al no existir VARCHAR(n) estricto, se usa TEXT con CHECK(length <= n); las fechas se almacenan en texto ISO-8601; el objeto JSON es TEXT validado con JSON1. Se activó PRAGMA foreign_keys=ON ANTES de iniciar la transacción y debe repetirse en cada conexión. Esta implementación se ejecutó realmente en SQLite 3.46.1.

9. Resultados observados y evidencia de pruebas

Ejecución SQLite 3.46.1 (en entorno local): 12 tablas, 58 columnas, 18 referencias FK verificadas por PRAGMA, un evento demostrativo con temperatura extraída como 24.1, y siete pruebas negativas rechazadas por el motor.

Caso probado en SQLite	Resultado
Objeto JSON inválido	Rechazado
JSON tipo arreglo	Rechazado
Fecha final anterior a la inicial	Rechazado
Comentario con FK inexistente	Rechazado
Correo institucional repetido	Rechazado
Nombre de área con longitud excesiva	Rechazado
Conexión del usuario consigo mismo	Rechazado

Evidencia: Evidencias/verificacion_local_sqlite.txt y Evidencias/red_social_pascualina_sqlite_demo.db. La construcción del script de PostgreSQL y MySQL se revisó de manera estática; no se presentan capturas ni pruebas exitosas inventadas de esos dos motores. Sus carpetas incluyen consultas de verificación para ejecutar cuando se disponga de los servidores.

10. Dificultades, limitaciones y propuestas de mejora

La dificultad técnica principal fue conservar la misma semántica entre gestores: el tratamiento de JSON, la validez de longitud en SQLite, el uso de zonas horarias, la activación de FK y la incompatibilidad de MySQL entre CHECK y acciones referenciales sobre los mismos campos. Se resolvieron con mapeos y comprobaciones específicas en vez de copiar literalmente el SQL de PostgreSQL.

Mejoras propuestas: ejecutar y documentar PostgreSQL y MySQL en sus versiones objetivo; ampliar pruebas de borrados en cascada/restricción; introducir migraciones versionadas; validar permisos y credenciales; aplicar normalización de correos desde aplicación; proteger datos de ubicación; definir retención de lecturas IoT; y evaluar índices por uso real antes de agregar costos de escritura.

11. Conclusión individual

Al pasar del modelo lógico a tres implementaciones, comprendí que portar una base de datos no consiste solo en cambiar nombres de tipos. Las reglas de integridad, los controles de JSON, el comportamiento de fechas y las claves foráneas tienen particularidades que pueden alterar el funcionamiento. La prueba en SQLite me permitió verificar que una definición DDL no basta por sí sola: también se necesitan casos válidos e inválidos. Considero PostgreSQL la opción más sólida para continuar el proyecto, especialmente por JSONB y sus controles, pero la comparación me ayudó a reconocer las ventajas de simplicidad de SQLite y la relevancia de MySQL en desarrollos web. Mi siguiente paso sería completar las ejecuciones reales de los dos motores pendientes y automatizar la verificación entre versiones.

12. Matriz de cumplimiento

Criterio	Estado	Observación
Trabajo individual y portada	Cumple	Solo Juan José Oquendo Sánchez
3 SGBD seleccionados y justificados	Cumple	PostgreSQL, MySQL y SQLite
Diccionario genérico actualizado	Cumple	XLSX completo con 58 campos
3 diccionarios físicos	Cumple	Tipos, claves y restricciones por motor
3 scripts DDL	Cumple (elaboración)	Se incluyen; solo SQLite probado en motor real
JSON / IoT	Cumple (diseño)	EVENTO.metadatos_contexto y prueba JSON SQLite
Memoria / informe técnico	Cumple	Este documento y archivos anexos
Ejecución real de 3 SGBD	Parcial	Solo SQLite comprobado
Conclusión individual	Cumple	Sección 11
Foro / retroalimentación	No acreditado	Debe realizarse externamente
Video de 5-10 min	No presentado	Excluido por decisión del autor
Subida a GitHub / campus	Pendiente	Los archivos quedan listos; no se publicaron

13. Referencias y material consultado

1. Proyecto Red Social Pascualina (2026). Tarea 1 - Modelo conceptual y propuesta consolidada (material elaborado previamente).

2. Proyecto Red Social Pascualina (2026). Tarea 2 - Modelo lógico normalizado y diccionario genérico (entregable previo).

3. Proyecto Red Social Pascualina (2026). Tarea 3 - Modelo físico PostgreSQL (entregable previo).

4. PostgreSQL Global Development Group. JSON Types. https://www.postgresql.org/docs/17/datatype-json.html

5. Oracle. MySQL 8.4 Reference Manual. The JSON Data Type. https://dev.mysql.com/doc/refman/8.4/en/json.html

6. Oracle. MySQL 8.4 Reference Manual. CHECK Constraints. https://dev.mysql.com/doc/refman/8.4/en/create-table-check-constraints.html

7. Oracle. MySQL 8.4 Reference Manual. DATE, DATETIME, and TIMESTAMP. https://dev.mysql.com/doc/refman/8.4/en/datetime.html

8. SQLite Consortium. JSON Functions and Operators. https://www.sqlite.org/json1.html

9. SQLite Consortium. PRAGMA statements. https://www.sqlite.org/pragma.html

10. Elmasri, R., & Navathe, S. A. (2007). Fundamentos de sistemas de bases de datos (5.ª ed.). Pearson. Bibliografía proporcionada en la actividad.